

Calculus Foundations for AI

The Mathematics of Machine Learning

Dr. Mustafa M. Shiple

Linear Algebra & Calculus Essentials

November 17, 2025

Presentation Outline

- 1 Introduction and Motivation
- 2 Univariate Differentiation (1D)
- 3 Multivariate Calculus: Partial Derivatives
- 4 The Gradient and Optimization
- 5 Chain Rule, Jacobian, and Backpropagation

3. Why Calculus in AI?

Calculus measures change. In AI, we must measure how small changes in inputs (like weights) affect outputs (like prediction error).

- **Rate of Change:** How fast is the error changing? (Derivative)
- **Direction:** Which direction reduces the error fastest? (Gradient)

4. The Goal: Minimize Error

All Machine Learning models aim to minimize a **Loss Function** L . This function quantifies the difference between the model's prediction and the true value.

$$\text{Loss} = L(\text{Weights})$$

The minimum of L represents the best possible model fit.

5. Calculus is the Engine of Learning

The primary algorithm used to find the minimum of the loss function is **Gradient Descent**.

- It uses the derivative (or gradient) to navigate the error landscape.
- It tells the model exactly how to update its internal parameters (weights) to reduce the error.

6. The Definition of a Derivative

The derivative $f'(x)$ is the instantaneous rate of change of a function $f(x)$.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

It represents the slope of the tangent line at point x .

7. Derivative as the Slope

For a linear function, the derivative is just the slope.

- **Function:** $y = 2x + 1$
- **Derivative:** $\frac{dy}{dx} = 2$
- **Interpretation:** For every 1-unit increase in x , y increases by 2 units.

8. Differentiation Rule 1: The Power Rule

If $f(x) = x^n$, then the derivative is:

$$f'(x) = nx^{n-1}$$

Numerical Example:

- $f(x) = x^3$
- $f'(x) = 3x^{3-1} = 3x^2$

9. Differentiation Rule 2: Constant Multiple Rule

If $f(x) = c \cdot g(x)$, then $f'(x) = c \cdot g'(x)$.

$$\frac{d}{dx}[c \cdot g(x)] = c \frac{d}{dx}[g(x)]$$

Numerical Example:

- $f(x) = 5x^4$
- $f'(x) = 5 \cdot (4x^3) = 20x^3$

10. Differentiation Rule 3: The Sum Rule

The derivative of a sum of functions is the sum of their derivatives.

$$\frac{d}{dx}[f(x) + g(x)] = f'(x) + g'(x)$$

Numerical Example:

- $f(x) = x^2 + 2x + 7$
- $f'(x) = 2x + 2 + 0$

11. Differentiation Rule 4: The Product Rule

If $h(x) = f(x)g(x)$, then $h'(x) = f'(x)g(x) + f(x)g'(x)$.

$$\frac{d}{dx}[f(x)g(x)] = f'g + fg'$$

Numerical Example:

- $h(x) = x \cdot e^x$
- $h'(x) = (1)e^x + x(e^x) = e^x(1 + x)$

12. Differentiation Rule 5: The Quotient Rule

If $h(x) = \frac{f(x)}{g(x)}$, then $h'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}$.

$$\frac{d}{dx} \left[\frac{f}{g} \right] = \frac{f'g - fg'}{g^2}$$

Algebraic Example:

- $f(x) = \frac{x}{x^2+1}$
- $f'(x) = \frac{1(x^2+1) - x(2x)}{(x^2+1)^2} = \frac{1-x^2}{(x^2+1)^2}$

13. Differentiation Rule 6: Exponential Function

The derivative of the natural exponential function.

$$\frac{d}{dx}[e^{f(x)}] = f'(x)e^{f(x)}$$

Numerical Example (with Chain Rule):

- $f(x) = e^{5x}$
- $f'(x) = e^{5x} \cdot (5) = 5e^{5x}$

14. Differentiation Rule 7: Natural Logarithm

The derivative of the natural logarithm function.

$$\frac{d}{dx}[\ln(f(x))] = \frac{1}{f(x)} f'(x)$$

Numerical Example (with Chain Rule):

- $f(x) = \ln(x^2)$
- $f'(x) = \frac{1}{x^2} \cdot (2x) = \frac{2}{x}$

15. Differentiation Rule 8: The Chain Rule (1D Simple)

Used for composite functions $f(g(x))$.

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$

Numerical Example:

- $y = (x^2 + 1)^3$
- $\frac{dy}{dx} = 3(x^2 + 1)^2 \cdot (2x) = 6x(x^2 + 1)^2$

41. The Multivariable Chain Rule (2D and more)

Consider $w = f(x(t), y(t))$

- $L \rightarrow \text{Output Layer} \rightarrow \text{Hidden Layer}_2 \rightarrow \text{Hidden Layer}_1 \rightarrow \text{Input}$
- We need to calculate $\frac{\partial L}{\partial w}$ for weights w deep inside the network.

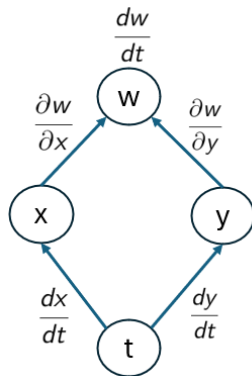
This requires the Multivariate Chain Rule.

42. Chain Rule (2D and more)

If $w = f(x, y)$ and $x = g(t)$, $y = h(t)$, then $\frac{dw}{dt}$ is the sum of changes through all paths:

This is the fundamental principle that drives the complex gradient calculation in neural networks.

$$\frac{dw}{dt} = \frac{\partial w}{\partial x} \frac{dx}{dt} + \frac{\partial w}{\partial y} \frac{dy}{dt}$$



16. Functions of Multiple Variables

In AI, the loss function L depends on many variables (weights, $w_1, w_2, \dots$ and biases, $b_1, b_2, \dots$).

$$L = f(w_1, w_2, b_1, \dots, w_n)$$

We need to measure the change with respect to **one variable at a time**.

17. Introducing Partial Derivatives

A partial derivative $\frac{\partial f}{\partial x}$ measures the rate of change of f along one axis (x), while holding all other variables constant.

- We use the symbol ∂ (curly d).
- This is key to determining each parameter's contribution to the total error.

18. First-Order Partial Definition ($\partial f / \partial x$)

The partial derivative with respect to x treats y as a constant.

$$\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h, y) - f(x, y)}{h}$$

All standard differentiation rules apply to x .

Function: $f(x, y) = x^2y + 3x + y^3$

- Treat y and y^3 as constants.
- The term y^3 differentiates to 0.
- **Result:** $\frac{\partial f}{\partial x} = 2xy + 3$

20. First-Order Partial Definition ($\partial f / \partial y$)

The partial derivative with respect to y treats x as a constant.

$$\frac{\partial f}{\partial y} = \lim_{k \rightarrow 0} \frac{f(x, y + k) - f(x, y)}{k}$$

All standard differentiation rules apply to y .

Function: $f(x, y) = x^2y + 3x + y^3$

- Treat x^2 and $3x$ as constants.
- The term $3x$ differentiates to 0.
- **Result:** $\frac{\partial f}{\partial y} = x^2(1) + 0 + 3y^2 = x^2 + 3y^2$

22. Interpretation: Holding Variables Constant

The partial derivative isolates the impact of one parameter.

- $\frac{\partial L}{\partial w_1}$: How much does the Loss L change if **only** weight w_1 changes by a small amount?
- This is essential in a neural network where thousands of weights are adjusted simultaneously.

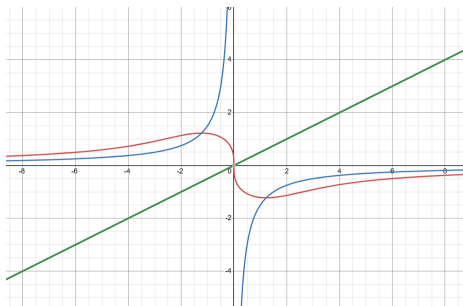
23. Second-Order Partial ($\partial^2 f / \partial x^2$)

Measures the **concavity** (curvature) of the function along the x-axis.

$$\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial x} \right)$$

Function: $f(x, y) = x^2y + 3x + y^3$

- Treat y and y^3 as constants.
- The term y^3 differentiates to 0.
- $\frac{\partial f}{\partial x} = 2xy + 3$
- **Result:** $\frac{\partial^2 f}{\partial x^2} = 2y$

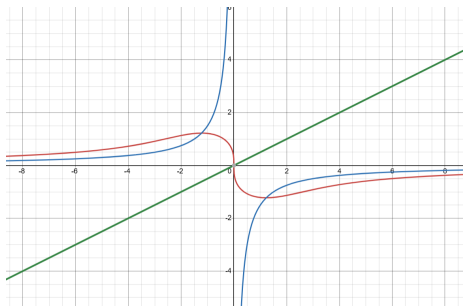


23. Second-Order Partial ($\partial^2 f / \partial x^2$)

Measures the **concavity** (curvature) of the function along the x -axis.

$$\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial x} \right)$$

- If $x > 0$. The function is concave up along the x -axis.
- If $x < 0$. The function is concave down along the x -axis.
- If $x = 0$. no curvature along the x -axis.



26. Introduction to the Gradient (∇f)

The Gradient is the central concept for optimization. It bundles all partial derivatives into a single vector.

- The gradient vector (∇f) points in the direction of the **greatest rate of increase** of f .
- Gradient Descent uses the negative gradient ($-\nabla f$) to find the minimum.

27. Gradient Definition (2D)

For a function $f(x, y)$, the gradient $\nabla f(x, y)$ is:

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix}$$

This is a vector containing the slopes in the x and y directions.

28. Gradient Definition (ND)

For a function of n variables, $f(x_1, x_2, \dots, x_n)$, the gradient vector is:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

In AI, n is the number of weights, often millions.

29. Gradient Example: Calculation

Function: $f(x, y) = x^2y + 3x^2$

- $\frac{\partial f}{\partial x} = 2xy + 6x$
- $\frac{\partial f}{\partial y} = x^2$

Gradient Vector:

$$\nabla f(x, y) = \begin{pmatrix} 2xy + 6x \\ x^2 \end{pmatrix}$$

30. Gradient Example: Evaluation at a Point

Evaluate the gradient of $f(x, y) = x^2y + 3x^2$ at the point $(1, 2)$.

- x-component: $2(1)(2) + 6(1) = 4 + 6 = 10$
- y-component: $(1)^2 = 1$

Result:

$$\nabla f(1, 2) = \begin{pmatrix} 10 \\ 1 \end{pmatrix}$$

31. Gradient Interpretation: Direction and Magnitude

For $\nabla f(1, 2) = \begin{pmatrix} 10 \\ 1 \end{pmatrix}$:

- **Direction:** The function increases fastest when moving in the ratio of 10 units in x for every 1 unit in y .
- **Magnitude:** $\|\nabla f\| = \sqrt{10^2 + 1^2} = \sqrt{101} \approx 10.05$. This is the maximum rate of increase.

32. The Direction of Steepest Ascent

The gradient vector $\nabla f(x_0)$ points directly toward the largest instantaneous increase in the function value from x_0 .

- In an error landscape, this is the direction of **maximum error increase**.
- We seek the opposite.

33. The Direction of Steepest Descent

To minimize the loss function, we move in the direction opposite to the gradient.

$$\text{Descent Direction} = -\nabla f$$

This is the core idea of **Gradient Descent**.

34. AI Insight: Error Surface Minimization

Imagine the Loss Function L as a mountainous terrain.

- The model is a hiker starting randomly.
- The gradient vector tells the hiker where the hill goes up (∇L).
- The hiker takes a step in the opposite direction ($-\nabla L$) to reach the valley (minimum error).

35. The Loss Function $L(w, b)$

Consider a simple model with one weight (w) and one bias (b). The Loss is $L(w, b)$.

$$L(w, b) = (\hat{y} - y_{\text{true}})^2$$

We need to calculate two partial derivatives to form the gradient:

- $\frac{\partial L}{\partial w}$
- $\frac{\partial L}{\partial b}$

36. Gradient of the Loss ∇L

The gradient of the loss function contains the change information for every parameter.

$$\nabla L = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_n} \end{pmatrix}$$

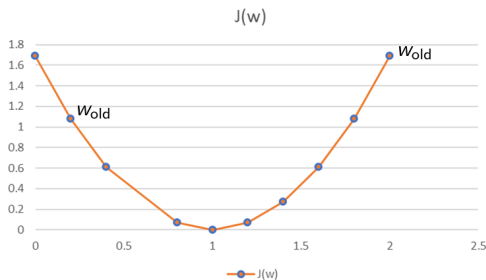
Each component is the sensitivity of the loss to that specific parameter.

37. Gradient Descent Step

The algorithm is iterative. At each step, we update the parameter w by subtracting a fraction of the partial derivative:

$$w_{\text{new}} = w_{\text{old}} - (\text{Learning Rate}) \times \left(\frac{\partial L}{\partial w} \right)$$

The Learning Rate (α) controls the size of the step.



38. Gradient Descent Update Rule (Vector Form)

For the full set of parameters $\theta = (w_1, \dots, b_n)$:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla L(\theta_{\text{old}})$$

Every parameter is updated simultaneously based on its unique partial derivative.

39. Numerical GD Step Example 1

Assume a simple function $L(w)$ (1D) and we start at $w = 5$.

- **Current Weight:** $w_{\text{old}} = 5$
- **Derivative (Slope):** $\frac{\partial L}{\partial w} = 10$ (Positive slope, we are going up)
- **Learning Rate:** $\alpha = 0.1$

Update:

$$w_{\text{new}} = 5 - 0.1(10) = 4.0$$

40. Numerical GD Step Example 2

Now we are at $w = 4.0$.

- **Current Weight:** $w_{\text{old}} = 4.0$
- **Derivative (Slope):** $\frac{\partial L}{\partial w} = 5$ (Still positive, but less steep)
- **Learning Rate:** $\alpha = 0.1$

Update:

$$w_{\text{new}} = 4.0 - 0.1(5) = 3.5$$

The step size naturally decreases as the slope flattens near the minimum.

41. The Multivariable Chain Rule

In a neural network, the loss function L is a composite function of many layers, with many variables.

- $L \rightarrow \text{Output Layer} \rightarrow \text{Hidden Layer}_2 \rightarrow \text{Hidden Layer}_1 \rightarrow \text{Input}$
- We need to calculate $\frac{\partial L}{\partial w}$ for weights w deep inside the network.

This requires the Multivariate Chain Rule.

42. Chain Rule for Backpropagation

If $w = f(x, y)$ and $x = g(t)$, $y = h(t)$, then $\frac{dw}{dt}$ is the sum of changes through all paths:

$$\frac{dw}{dt} = \frac{\partial w}{\partial x} \frac{dx}{dt} + \frac{\partial w}{\partial y} \frac{dy}{dt}$$

This is the fundamental principle that drives the complex gradient calculation in neural networks.

43. Backpropagation: Calculation of Error Gradients

Backpropagation (Backprop) is the specific application of the Multivariate Chain Rule in neural networks.

- It calculates the gradient of the loss function with respect to every weight.
- It works **backward** from the final error (Loss) to the initial weights.

44. Backpropagation: Layer-by-Layer

To find the change in Loss L with respect to a weight w in Layer 1:

$$\frac{\partial L}{\partial w_{\text{Layer 1}}} = \left(\frac{\partial L}{\partial \text{Layer 2}} \right) \times \left(\frac{\partial \text{Layer 2}}{\partial \text{Layer 1}} \right) \times \left(\frac{\partial \text{Layer 1}}{\partial w_{\text{Layer 1}}} \right)$$

The chain rule links the impact of the final error back through all preceding layers.

45. Simple Backprop Example (Algebraic)

Let the composite function be $z = y^2$ and $y = \cos(x)$. We want $\frac{dz}{dx}$.

- Outer derivative: $\frac{dz}{dy} = 2y$
- Inner derivative: $\frac{dy}{dx} = -\sin(x)$

Chain Rule:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} = 2y(-\sin(x)) = -2 \cos(x) \sin(x)$$

46. Simple Backprop Example (Numerical)

Let $z = y^2$ and $y = x + 1$. Find $\frac{dz}{dx}$ at $x = 1$.

- At $x = 1$, $y = 1 + 1 = 2$.
- $\frac{dz}{dy} = 2y = 2(2) = 4$
- $\frac{dy}{dx} = 1$

Result:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} = 4 \cdot 1 = 4$$

47. The Jacobian Matrix (Concept)

The Jacobian is the ultimate generalization of the derivative for a vector-valued function.

- If a function $\mathbf{f}$ maps a vector $\mathbf{x}$ to another vector $\mathbf{y}$, the Jacobian J is the matrix of all first-order partial derivatives.
- In neural networks, the Jacobian describes the transformation from one layer's vector of activations to the next.

48. The Jacobian Matrix Definition

For $\mathbf{f}(x_1, x_2)$ with outputs $\mathbf{y} = (y_1, y_2)$:

$$J = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{pmatrix}$$

The full Chain Rule for vector functions uses matrix multiplication of Jacobians.

49. Conclusion: Calculus Enables Learning

Calculus provides the mathematical tools to enable machines to learn by:

- **Quantifying Error Change:** Partial derivatives.
- **Finding the Path to Minimum:** The negative gradient.
- **Linking Layers:** The Multivariable Chain Rule (Backprop).

50. Summary of Key Concepts

- **Derivative** ($\frac{df}{dx}$): Rate of change in 1D.
- **Partial Derivative** ($\frac{\partial f}{\partial x}$): Rate of change w.r.t. one variable, others constant.
- **Gradient** (∇f): Vector of all partial derivatives; points to steepest ascent.
- **Gradient Descent**: Iterative optimization using $-\nabla f$.
- **Chain Rule (Backprop)**: Method for computing gradients across layered functions.